

PROJECT REPORT

Red-Black Tree (RBT) Implementation

Developed in: C Programming Language

Architecture: Command-Line Interface (CLI)

Core Concept: Advanced Self-Balancing Data Structures

1. INTRODUCTION

While standard Binary Search Trees (BSTs) provide efficient data retrieval in optimal scenarios, they suffer from a critical flaw: sequential data insertions cause the tree to degrade into a skewed linked list, resulting in a disastrous $O(N)$ time complexity. A Red-Black Tree resolves this vulnerability by enforcing strict color-coded topological rules, establishing a self-balancing architecture. The Red-Black Tree Implementation project is a C-based software utility engineered to construct, manipulate, and balance these complex trees from scratch, guaranteeing $O(\log N)$ performance for all search, insertion, and deletion operations.

2. SYSTEM ARCHITECTURE & MODULES

The system utilizes a modular C architecture, isolating operational algorithms into dedicated source files to ensure maintainability and algorithmic clarity.

2.1. Execution & Routing (main.c)

This module acts as the interactive driver program. It maintains the root pointer of the tree and exposes a menu-driven CLI. It routes user inputs to corresponding algorithmic files, including insertion, deletion, and finding/deleting extremums (minimum and maximum nodes), followed by triggering a visual representation of the updated tree topology.

2.2. Insertion & Rotation Engine (insert.c)

This module handles standard BST insertion, automatically coloring new nodes Red. It subsequently launches a validation loop to detect Red-Red violations. Depending on the color of the "Uncle" node, the engine executes localized recoloring or triggers structural realignment via `left_rotate()` and `right_rotate()` functions to maintain the Black-Height property.

2.3. Deletion & Resolution Engine (delete.c)

The deletion module is the most structurally complex component, designed to handle the notorious "Double Black" problem. It resolves black-depth deficits across six distinct topological edge cases, dynamically shifting colors and executing sibling rotations until the tree's structural integrity is restored.

3. DATA STRUCTURES & ALGORITHMIC OPTIMIZATIONS

The foundation of the project relies on a specialized node structure designed to track data and topological metadata simultaneously.

Node Structure

```
typedef struct tree_node {  
    int data;                // Payload  
    int color;               // 1 for RED, 0 for BLACK  
    struct tree_node *left;  // Left child pointer  
    struct tree_node *right; // Right child pointer  
} tree_t;
```

Payload Swapping vs. Pointer Rewiring

A standard approach to deleting a node with two children involves extreme pointer manipulation, rewiring up to six different connections. To mitigate the risk of dangling pointers and segmentation faults, this project employs an $O(1)$ payload swap. The algorithm simply overwrites the target node's data with its in-order successor's data, preserving the complex structural topology of the tree, and subsequently deletes the simpler leaf node.

4. KEY FEATURES & OPERATIONS

Feature	Implementation Details
Double Black Resolution	When a Black node is deleted, the branch suffers a black-depth deficit. The algorithm propagates this "Double Black" status up the tree, utilizing sibling properties (color and children) to resolve the deficit via structural rotations or recoloring.
Extremum Operations	Implements dedicated modules to find and delete the minimum/maximum values, heavily optimizing operations like priority queue polling.
Dynamic Topological Visualizations	Includes a recursive <code>print_tree()</code> utility that traverses the active structure, providing immediate command-line feedback regarding node relationships and color assignments.

5. LIMITATIONS AND FUTURE ENHANCEMENTS

While the implementation correctly balances complex data insertions, there are critical software engineering enhancements required for production-level stability:

- **Heap Memory Validation:** The insertion algorithm currently invokes `malloc()` without validating the return pointer. In memory-constrained environments, an allocation failure will return `NULL`, resulting in an immediate Segmentation Fault upon dereferencing. Adding `if (new_node == NULL)` checks is mandatory for resilience.
- **Input Sanitization:** The `main.c` loop utilizes `scanf("%d")`. If a user provides an alphabetical character, the unconsumed buffer creates an infinite operational loop. Implementing buffer flushing mechanisms (`while(getchar() != ' ')`) will harden the CLI against malformed inputs.
- **Global Memory Reclamation:** Although the deletion module correctly `free()`s targeted nodes, terminating the program via the menu does not clear the remaining tree structure. A `destroy_tree()` function executing a post-order traversal must be implemented to free all active nodes and prevent memory leaks upon exit.

6. CONCLUSION

The Red-Black Tree Implementation project is a rigorous exercise in advanced algorithm design and multi-level pointer manipulation. By successfully architecting dynamic structural rotations and edge-case resolution matrices (such as Double Black handling), the project demonstrates a profound capability to develop high-performance, self-optimizing data structures natively in C.